Machine Learning, Chapter 2: Regression revision (slides 14–63)

Course: Dr. Ameni Youssfi Nouira · PRISTINI 2026/2027 · Source: `Chap2_Supervised_Learning_Regression.pdf`. Every number was re-checked in Python. ⚠ marks a mistake on the slide; **EXAM** marks what you need for the exam.

Words used everywhere

Symbol	Meaning
x	input / feature (e.g. house size)
y	**real** answer from the data (the real price)
$\hat{y} = f_{w,b}(x) = h_\theta(x)$	**predicted** answer, what the line says (three names, one thing)
w, b (or β0, β1…)	**parameters**: the numbers the model *learns*
m	number of examples (rows)
J	cost: one score for how bad the line is. **Smaller is better; 0 is perfect** (J is not a percentage)

Part A: Slides 14–15, what "predicted" means (the house example)

The model doesn't *predict* w and b. It **learns** them from the data, then uses them to **predict prices**.

Step 1: the data (what we already know)

House	Size x	Real price y
A	1	2
B	2	4
C	3	5

Step 2: training means finding w and b. The computer tries a line, e.g. **w = 1.5, b = 0.5**, so $\hat{y} = 1.5x + 0.5$.

Step 3: predicted = what the line says the price is

House	x	Real y	Predicted $\hat{y} = 1.5x + 0.5$	Error $\hat{y} - y$
A	1	2	$1.5 \cdot 1 + 0.5 =$ **2**	0
B	2	4	$1.5 \cdot 2 + 0.5 =$ **3.5**	−0.5
C	3	5	$1.5 \cdot 3 + 0.5 =$ **5**	0

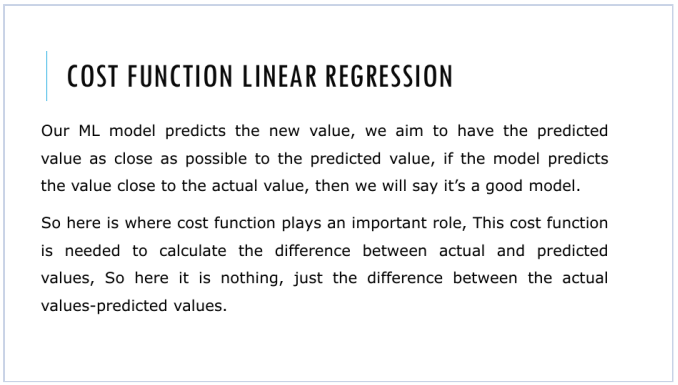
Real y comes from the data; **predicted ŷ** is the line's guess. On the graph, y is the dot and ŷ is the point on the line directly above or below it.

Step 4: the cost J gives one score for this line (slide 14) $J = 1/(2 \cdot 3) \cdot (0^2 + 0.5^2 + 0^2) = 0.25/6 =$ **0.04**, close to 0, so a good line. A bad line (w = 0, b = 0) gives $J = 1/6 \cdot (4 + 16 + 25) =$ **7.5**. The computer keeps changing w and b (gradient descent) and keeps the values with the **smallest** J.

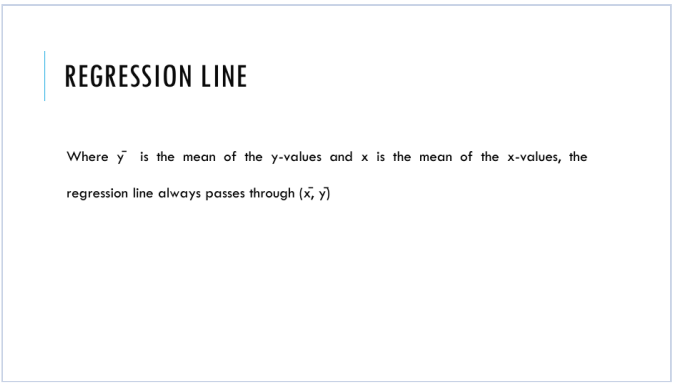
Step 5: use the line on a new house. Size x = 4 isn't in the data → ŷ = 1.5·4 + 0.5 = **6.5**. That's the real purpose.

Slide 15: the **best** line always passes through the average point **($\bar{x}$, $\bar{y}$)**. Here (2, 3.67); our line gives 3.5 there, so it's close to the best but not exactly it.

The whole process: data (x, y) → try w, b → predict ŷ → J measures the errors → adjust w, b to make J smaller → repeat → use the line on new houses.



Slide 14



Slide 15

⚠ Slide 14 says "as close as possible to the **predicted** value". It should read the **actual** value.

EXAM **Exam:** cost = actual vs predicted; the best line passes through ($\bar{x}$, $\bar{y}$).

Part B: Slides 16–37, cost function and gradient descent

Slide 16: types of cost function (mean error vs MSE)

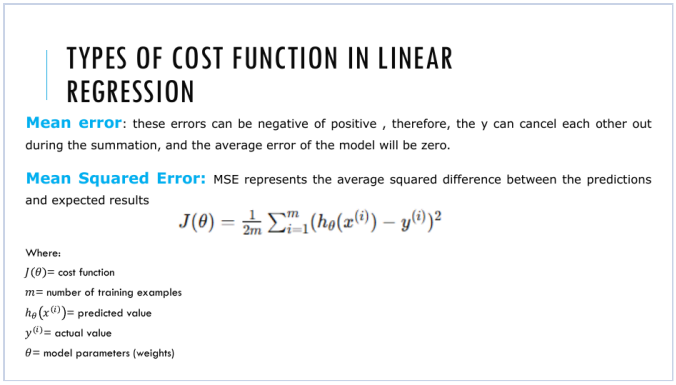
- **Mean error:** add the errors and average them. +3 and −3 cancel to 0, so a terrible model can show "0 error". Useless on its own.
- **MSE (the course's cost): $J(\theta) = \frac{1}{2m} \cdot \Sigma\, (h_\theta(x^{(i)}) - y^{(i)})^2$**. Squaring makes every error positive, so nothing cancels. $h_\theta(x)$ is just the prediction ŷ, and θ means all the parameters (w and b).

EXAM Explain **why we square**; know the symbols (m, $h_\theta(x^{(i)})$, $y^{(i)}$).

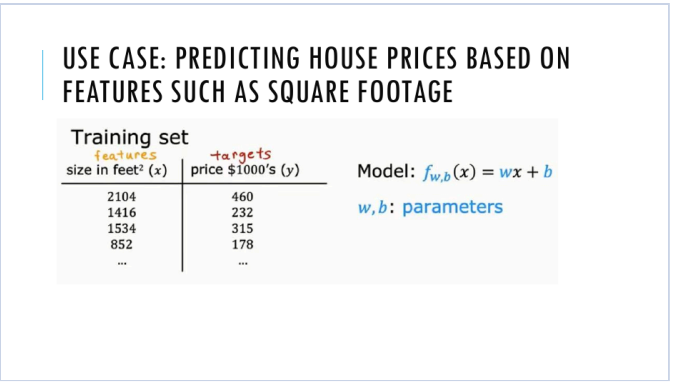
Slide 17: use case, house prices (training-set picture)

size in feet² (x) = feature	price in $1000's (y) = target
2104	460
1416	232
1534	315
852	178

Model $\mathbf{f_{w,b}(x) = wx + b}$; **w, b = parameters**. Feature = the input you know; target = the answer to predict; parameters = the knobs the model adjusts.



Slide 16



Slide 17

EXAM Given a table, name the feature, the target and the parameters.

Slides 18–21: cost function with b = 0 (the key visual idea)

Slide 18: red crosses = data, blue line = model. For one point $x^{(i)}$: the cross is the real $y^{(i)}$, the circle on the line is the predicted $\hat{y}^{(i)}$, and **the vertical gap is the error**. Goal: find w, b so that $\hat{y}^{(i)}$ is close to $y^{(i)}$ for all examples.

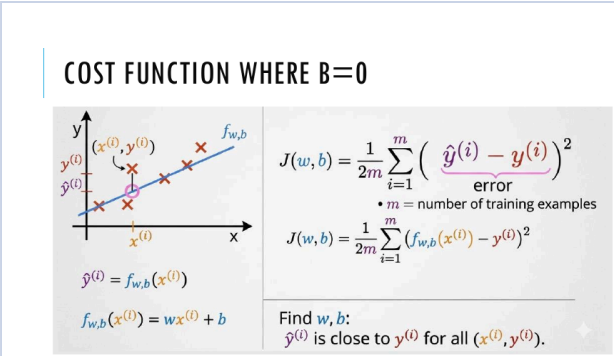
Slide 19: simplify by setting **b = 0**, so J can be drawn as a simple 2D curve:

General	Simplified (b = 0)
f(x) = wx + b	f(x) = wx (line through the origin)
parameters w, b	parameter w only
minimize J(w,b)	minimize J(w)

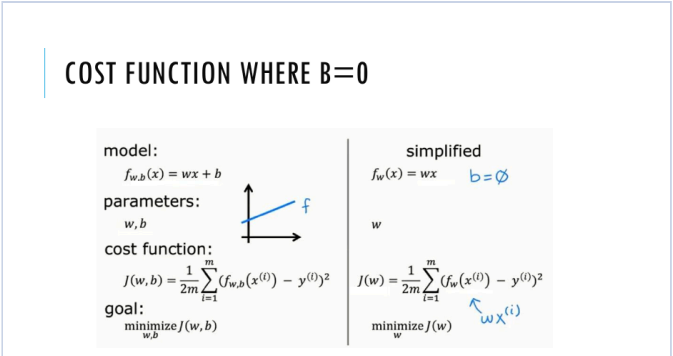
Slide 20: data (1,1), (2,2), (3,3). Try different values of w and compute J for each ($\hat{y}$ = wx, m = 3):

w	predictions $\hat{y}$	errors $\hat{y} - y$	J(w) = 1/6 · Σ(error²)
1	1, 2, 3	0, 0, 0	**0** ✓
0.5	0.5, 1, 1.5	−0.5, −1, −1.5	1/6·(0.25 + 1 + 2.25) = **0.58**
0	0, 0, 0	−1, −2, −3	1/6·(1 + 4 + 9) = **2.33**
−0.5	−0.5, −1, −1.5	−1.5, −3, −4.5	1/6·(2.25 + 9 + 20.25) = **5.25**

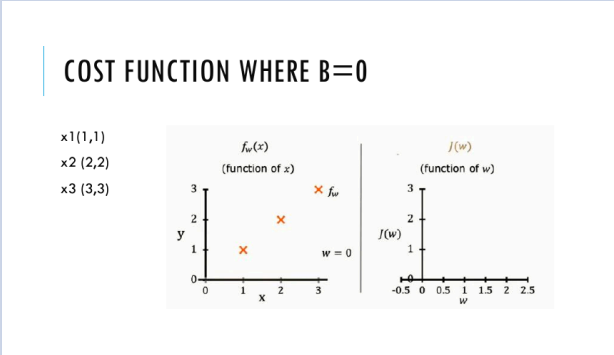
Slide 21: plotted, these points form a **U-shaped curve (bowl)** with its bottom at **w = 1, J = 0**. "Choose w to minimize J(w)." Each w is a different line; the best line is at the bottom of the U.



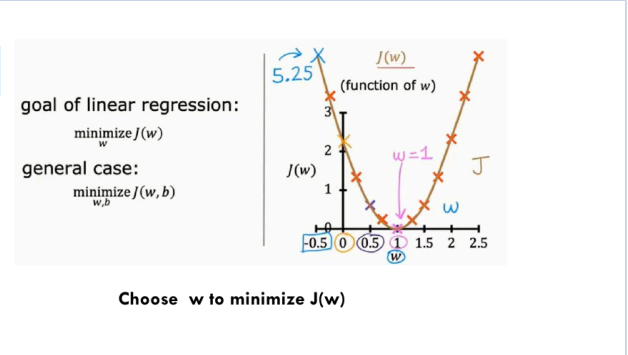
Slide 18



Slide 19



Slide 20

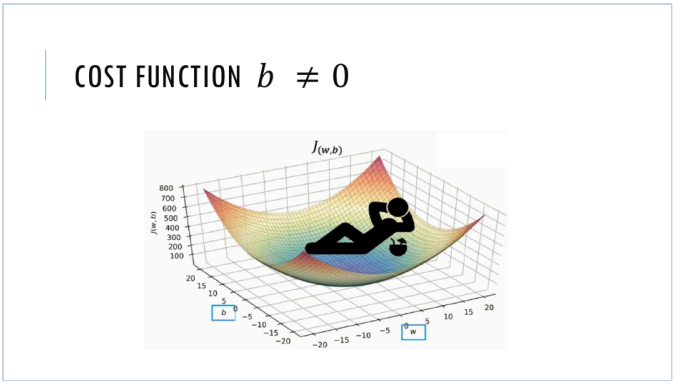


Slide 21

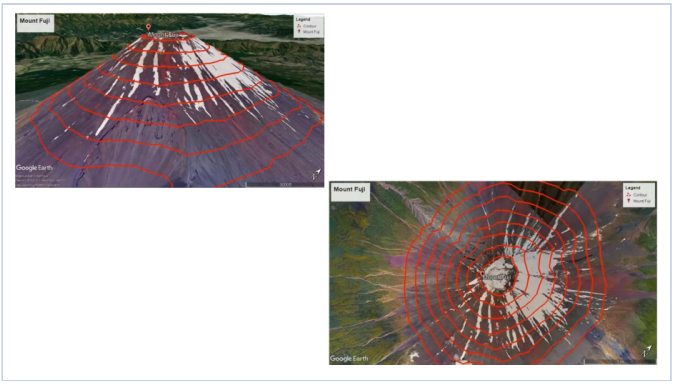
EXAM **Very likely:** compute J(w) by hand for a few values of w and explain that the minimum of J gives the best line.

Slides 22–26: cost with b ≠ 0, and contour maps

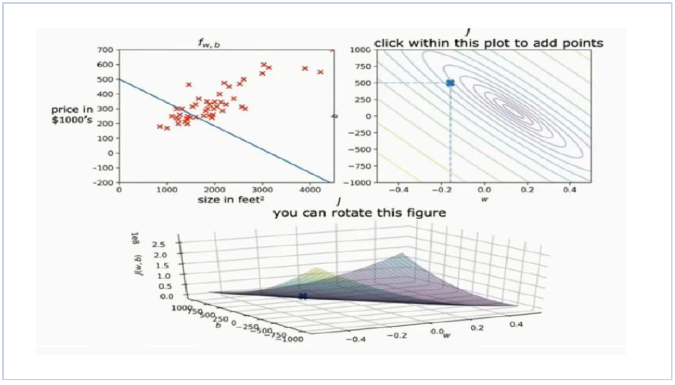
- **Slide 22:** "See Google Colab" (the live demo in class).
- **Slide 23 (3D bowl):** with both w and b, J(w,b) is a **3D bowl**: w and b on the floor, J as the height. The person lying at the bottom is the minimum, the best (w, b).
- **Slide 24 (Mount Fuji):** red rings = **contour lines**, points at the **same height**. Seen from above, they're circles inside circles. A **contour plot** is the bowl seen from above: each ellipse is a set of (w, b) pairs with the **same J**, and the centre is the minimum.
- **Slide 25 (bad model):** the line with w ≈ −0.15, b ≈ 500 slopes down while the data goes up. Its × is **far from the centre**, so J is high.
- **Slide 26 (good model): f(x) = 0.13x + 71** has small error arrows, and its × is at the **centre** of the ellipses (w = 0.13, b = 71).
- **One point on the contour plot = one line on the data graph.** Close to the centre = a good line.



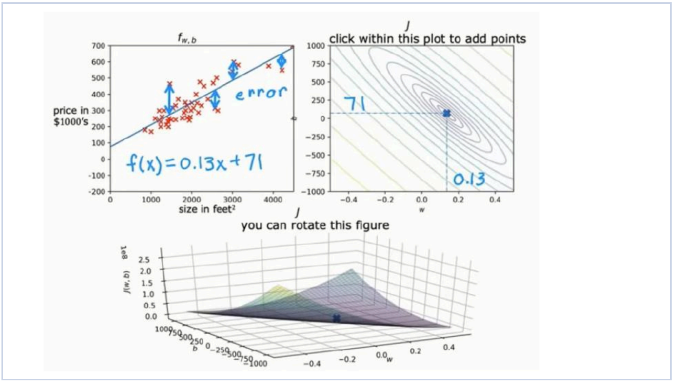
Slide 23



Slide 24



Slide 25



Slide 26

EXAM Contour plot = bowl seen from above, rings of equal J, centre = best (w, b).

Slides 27–29: why gradient descent?

- **Slide 28:** the cost function tells you **how good or bad** the model is, not **how to improve it**. Gradient descent gives the **direction** to change w and b. **J measures; gradient descent improves.**
- **Slide 29:** (1) start with some w, b (often 0, 0); (2) keep changing them to reduce J; (3) stop at or near a minimum.

EXAM The difference between the cost function and gradient descent is a classic short question.

Slide 30: a bumpy J surface (local minima)

A 3D surface with **several hills and valleys**. Gradient descent rolls into the nearest valley, which may not be the deepest: a **local minimum** vs the **global minimum**. **(extra)** For linear regression with MSE, J is always one single bowl (convex), so it always finds the best answer.

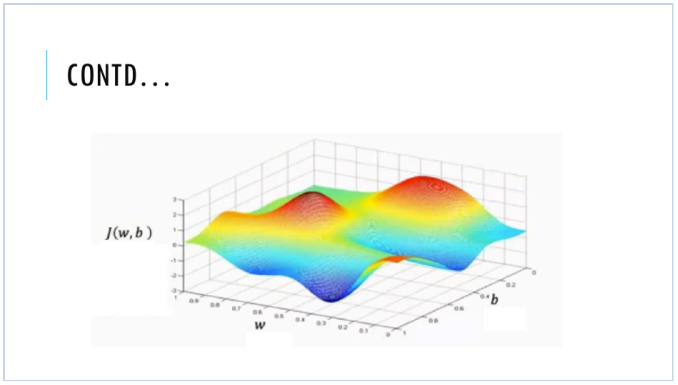
Slide 31: the gradient descent algorithm

```
Repeat until convergence:
  w = w − α · ∂J(w,b)/∂w
  b = b − α · ∂J(w,b)/∂b
```

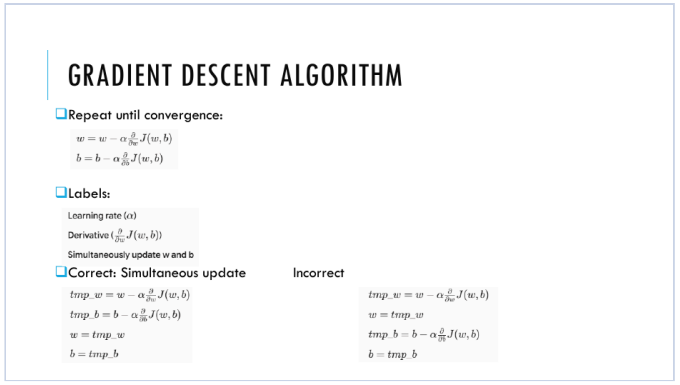
α = learning rate (step size); ∂J/∂w = derivative (the slope); **update w and b simultaneously**.

✓ **Correct (simultaneous)**	✗ **Incorrect**
tmp_w = w − α·∂J/∂w	tmp_w = w − α·∂J/∂w
tmp_b = b − α·∂J/∂b	**w = tmp_w** ← w changed too early
w = tmp_w	tmp_b = b − α·∂J/∂b ← uses the NEW w ✗
b = tmp_b	b = tmp_b

Compute **both** new values from the **same old position**, then move.



Slide 30



Slide 31

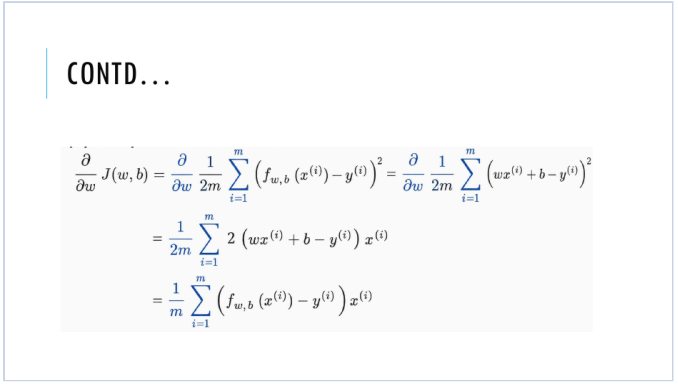
EXAM **Very likely:** spot which version is correct and explain why.

Slides 32–33: where the derivative formulas come from

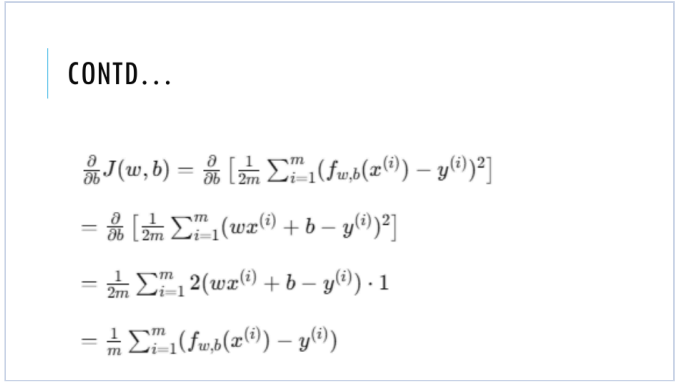
```
∂J/∂w = ∂/∂w [ 1/(2m) · Σ (wx⁽ⁱ⁾ + b − y⁽ⁱ⁾)² ]
      = 1/(2m) · Σ 2·(wx⁽ⁱ⁾ + b − y⁽ⁱ⁾) · x⁽ⁱ⁾     (chain rule: inner derivative = x)
      = 1/m · Σ (f(x⁽ⁱ⁾) − y⁽ⁱ⁾) · x⁽ⁱ⁾             (the 2 cancels the ½)

∂J/∂b = 1/m · Σ (f(x⁽ⁱ⁾) − y⁽ⁱ⁾)                    (inner derivative w.r.t. b = 1)
```

That's **why the ½ is in J**: it cancels the 2. The only difference between the two: **∂J/∂w has "× x", ∂J/∂b doesn't.**



Slide 32



Slide 33

EXAM Memorize both final formulas. Re-deriving them with the chain rule can earn bonus marks.

Slide 34: positive vs negative slope (two pictures)

- **Top:** right side of the U. Tangent goes up (slope +2), so **∂J/∂w > 0** → w − α·(positive) → **w decreases**: move left, toward the bottom.
- **Bottom:** left side. Slope −2, so **∂J/∂w < 0** → w − α·(negative) → **w increases**: move right, toward the bottom.
- The minus sign always sends you **downhill**. At the bottom the slope = 0 and w stops changing: **convergence**.

EXAM "If ∂J/∂w > 0, what happens to w?" It decreases. Explain why.

Slides 35–37: the full worked example (w = 0, b = 0, α = 0.1)

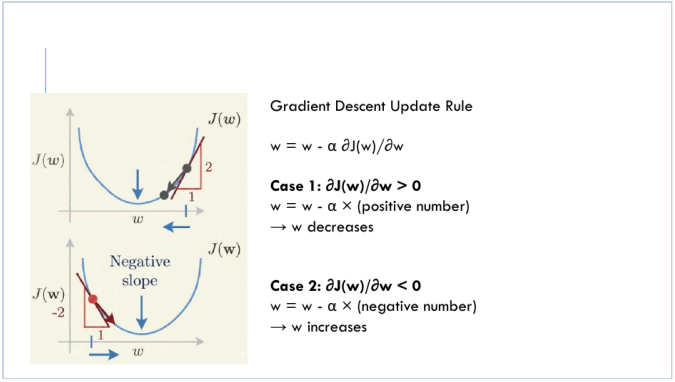
Data (1,2), (2,4), (3,5), (4,8), m = 4.

1. Predictions with w = b = 0: all **0**. Errors ŷ − y: **−2, −4, −5, −8**.

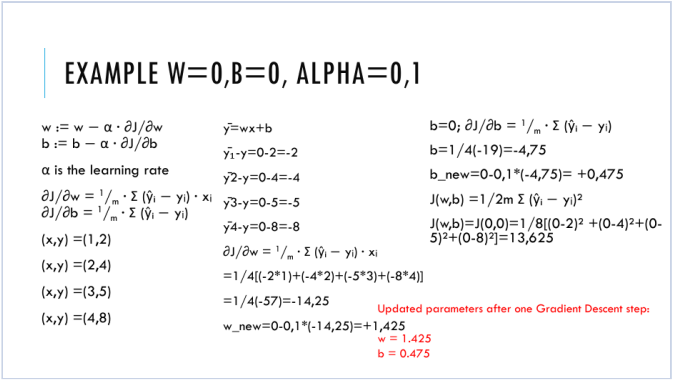
2. J(0,0) = 1/8 · (4 + 16 + 25 + 64) = 109/8 = **13.625**
3. ∂J/∂w = 1/4 · (−2·1 − 4·2 − 5·3 − 8·4) = −57/4 = **−14.25**
4. ∂J/∂b = 1/4 · (−2 − 4 − 5 − 8) = −19/4 = **−4.75**
5. w = 0 − 0.1·(−14.25) = **1.425**; b = 0 − 0.1·(−4.75) = **0.475** (negative derivatives, so both go up: the slide 34 rule)

Slide 36: new line ŷ = 1.425x + 0.475 → predictions 1.9, 3.325, 4.75, 6.175 → errors −0.1, −0.675, −0.25, −1.825 J = 1/8 · (0.01 + 0.4556 + 0.0625 + 3.3306) = **0.4823**

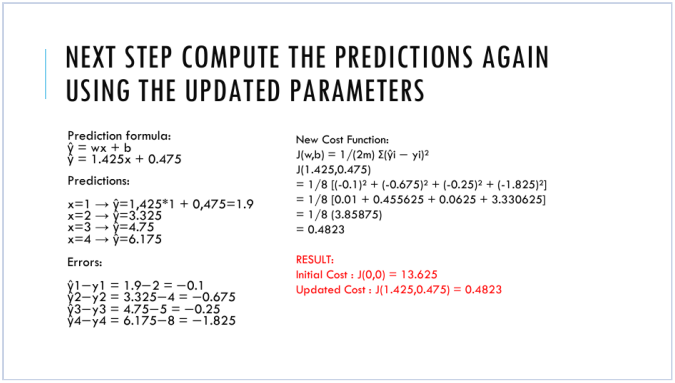
Slide 37: iteration 0: J = 13.625 → iteration 1: J = 0.4823 → iteration 2: blank on the slide, so practise it. The cost went down, so the line got better. Repeat until J stops decreasing.



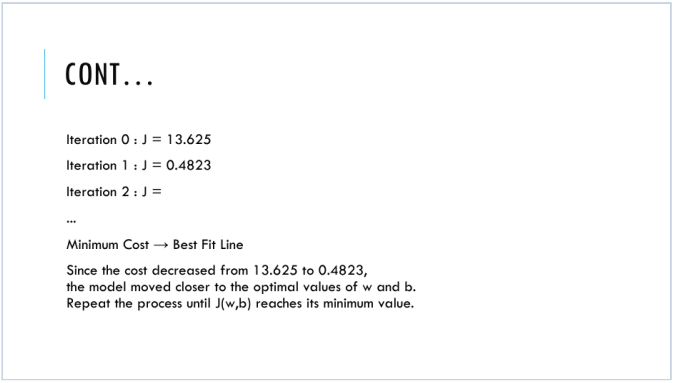
Slide 34



Slide 35



Slide 36



Slide 37

Practice: iteration 2. Start from w = 1.425, b = 0.475 and compute the new w, b and J yourself. Answer at the end of this document.

EXAM **Almost certain:** do 1–2 iterations by hand exactly like slides 35–36.

Part C: Slides 38–41, learning rate and error measures

Slide 38: how to choose the learning rate α

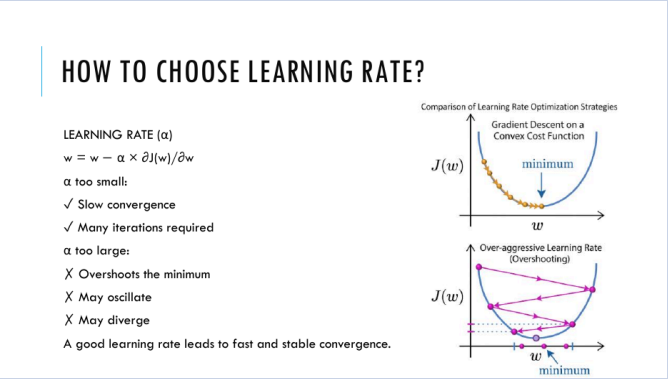
`w = w − α · ∂J/∂w`, so **α = the size of each step.**

- **Top picture (good α):** orange dots walk **down** the U in small steps to the minimum.
- **Bottom picture (too large, "over-aggressive"):** pink arrows **jump from side to side** and climb higher each time. That's overshooting.

α	What happens
too small	✓ works but **slowly**: many iterations
too large	✗ **overshoots**, ✗ may **oscillate**, ✗ may **diverge** (J explodes)
good	fast **and** stable convergence

Example (the professor's data, starting from w = b = 0). J over the first 4 iterations:

α	iter 0	iter 1	iter 2	iter 3	Verdict
0.01	13.6	11.5	9.6	8.1	too slow
0.1	13.6	0.48	0.12	0.11	good ✓
0.3	13.6	30.7	69.4	157	diverges



Slide 38

EXAM "α too large / too small?" with a sketch. "J increases after each iteration, why?" → α is too large.

Slide 39: MAE (Mean Absolute Error)

MAE = 1/m · Σ |ŷ − y|, the average **distance**, with no squaring.

⚠ **Slide error:** "MAE is a variation of MAE that squares the difference instead of taking the absolute value" is **wrong**. **MAE = absolute value |…|; MSE = square.** The formula on the slide is correct; only the sentence is wrong. Use the formula.

Slide 40: RMSE (Root Mean Squared Error)

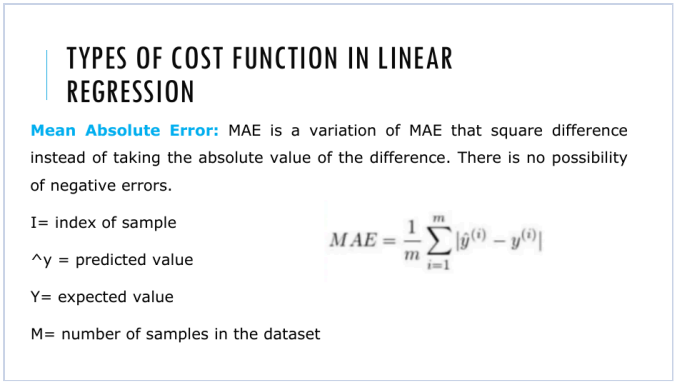
RMSE = √(Σ (predicted − actual)² / N) = √MSE. It brings the error back to the **same unit as y** (k$ instead of k$²). One of the two main performance measures for regression.

The three measures on the same example

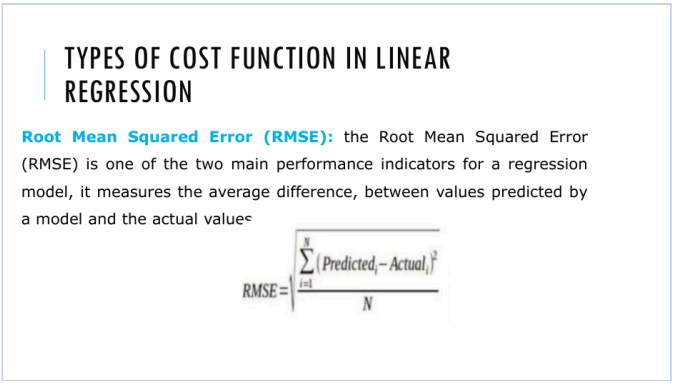
Real y = 10, 20, 30, 40; predicted ŷ = 12, 18, 31, 30. Errors: +2, −2, +1, **−10** (one big mistake).

Measure	Calculation	Result
MAE	(2 + 2 + 1 + 10) / 4	**3.75**
MSE	(4 + 4 + 1 + 100) / 4	**27.25**
RMSE	√27.25	**5.22**

The single −10 gives 100 of the 109 in MSE: squaring punishes big errors hard. **MSE and RMSE are sensitive to outliers; MAE is more robust.** (The course's J = 1/(2m)·Σ(…)² = MSE ÷ 2.)



Slide 39



Slide 40

EXAM Compute MAE / MSE / RMSE on 3–4 points; which is most sensitive to outliers (MSE); why RMSE is easier to interpret.

Slide 41, "LAB datamining": your **Lab 1** (load, clean, scale, 70/30 split, train, compute MSE/RMSE).

Part D: Slides 42–52, multiple linear regression

Slide 42: the equation

Y = β0 + β1·X1 + β2·X2 + … + ε. Y = dependent (response); X1, X2… = independent (predictors); **β0** = y-intercept; **β1, β2…** = slope coefficients, one per input; **ε** = error term.

Slide 43: table with 4 features

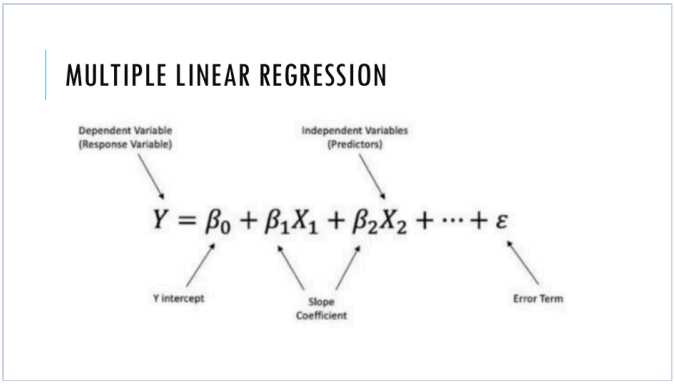
Size (feet²) x1	Bedrooms x2	Floors x3	Age x4	Price ($1000's) y
2104	5	1	45	460
1416	3	2	40	232
1534	3	2	30	315
852	2	1	36	178

Slide 44: the model with an example

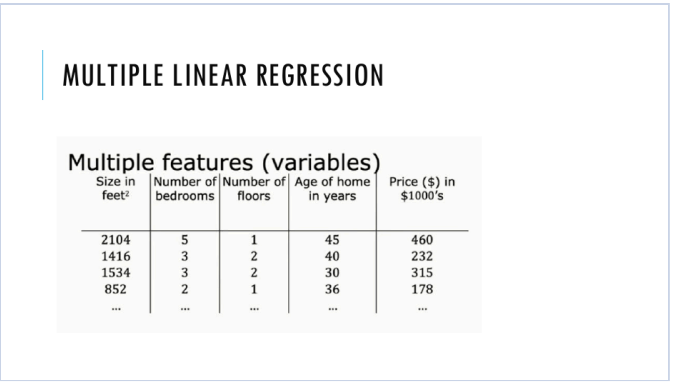
Before: f(x) = wx + b. Now: **f(x) = w1x1 + w2x2 + w3x3 + w4x4 + b**. Example: **f(x) = 0.1·x1 + 4·x2 + 10·x3 − 2·x4 + 80**

Reading it: +0.1 per feet², +4 per bedroom, +10 per floor, **−2 per year of age** (an older house is cheaper: a negative coefficient), 80 = base price. Each one applies **while the other inputs stay the same**.

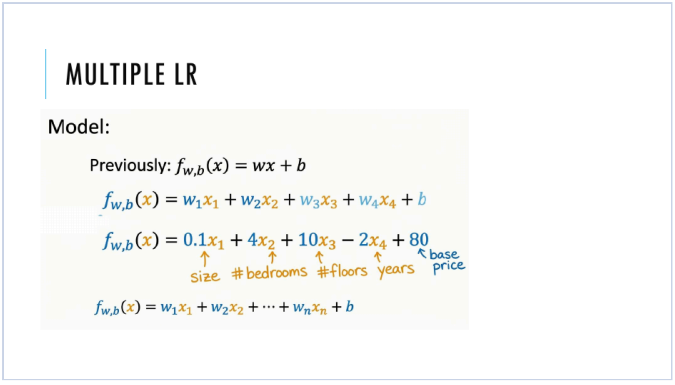
House 1 (2104, 5, 1, 45): 210.4 + 20 + 10 − 90 + 80 = **230.4 k$** (real: 460, so this example model under-predicts; training would fix the w's).



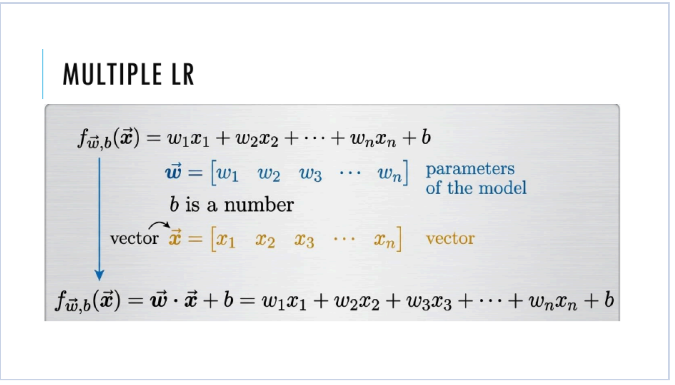
Slide 42



Slide 43



Slide 44



Slide 45

Slide 45: vector notation

$\vec{w}$ = [w1 … wn] (parameters), **$\vec{x}$ = [x1 … xn]** (inputs), b = a number. **$f(\vec{x}) = \vec{w} \cdot \vec{x} + b$**, where $\vec{w} \cdot \vec{x}$ = w1x1 + … + wnxn (the **dot product**). In NumPy: `np.dot(w, x) + b`.

Slides 46–47: two ways to find the β's

- **Gradient descent:** βj := βj − α · ∂J/∂βj for **every** j, repeated until J is minimal.
- **Normal equation (analytical): $\beta = (X^TX)^{-1} X^TY$**, the best β's directly in one formula.

	Gradient descent	Normal equation
How	step by step	one formula
Needs α?	yes	no
Big data	✓ fast	✗ slow (inverting a big matrix)
Result	approximate	exact

Slides 48–50: worked example (weekly sales)

Y = weekly sales, x1 = Product 1, x2 = Product 2: **Y = β0 + β1x1 + β2x2**

x1	x2	Y
1	4	1
2	5	6
3	8	8
4	2	12

Step 0: build X with a **column of 1s** in front (it multiplies β0). The slide's "114, 125, 138, 142" are the rows [1 1 4], [1 2 5], [1 3 8], [1 4 2].

Step 1: X^TX (each entry is a sum):

```
XᵀX = [ 4  10  19]    4 = rows   10 = Σx1    19 = Σx2
      [10  30  46]   30 = Σx1²  46 = Σx1·x2
      [19  46 109]  109 = Σx2²
```

Check: Σx1·x2 = 1·4 + 2·5 + 3·8 + 4·2 = 46 ✓

Step 2: inverse

```
(XᵀX)⁻¹ ≈ [ 3.15  −0.59  −0.30 ]
          [−0.59   0.20   0.016]
          [−0.30   0.016  0.054]
```

Step 3: multiply by Xᵀ

```
(XᵀX)⁻¹Xᵀ ≈ [ 1.36   0.47   −1.02   0.19 ]
            [−0.32  −0.098   0.155  0.26 ]
            [−0.065  0.005   0.185 −0.125]
```

⚠ **Slide error:** the first entry is printed as **0.05**. It's really **1.36** (with 0.05, β0 would be about −3.0). The slide's final β's are correct.

Step 4: multiply by Y. β0 = 1.36·1 + 0.47·6 − 1.02·8 + 0.19·12 ≈ **−1.69**, and the same pattern gives **β1 = 3.48, β2 = −0.05**.

Result: Y = −1.69 + 3.48·x1 − 0.05·x2 (⚠ the slide writes "x4" at the end: a typo for **x2**). Each extra unit of Product 1 adds ≈ 3.48 to sales; Product 2 barely matters. Check row 1: −1.69 + 3.48 − 0.20 = **1.57** vs real 1.

In the exam: a 3×3 inverse by hand is long. You'll likely be given $(X^TX)^{-1}$, or asked to build X with the 1s column and compute X^TX.

MULTIPLE LR

1.Gradient Descent

We update parameters step by step:

$\beta_j := \beta_j - \alpha \frac{\partial J}{\partial \beta_j}$

Repeated until the cost function is minimized.

✓ Works for big datasets
✓ Used in machine learning models

Slide 46

MULTIPLE LR

2. Analytical solution

$$\beta = (X^T X)^{-1} X^T y$$

This directly gives optimal values of β

✓ Exact solution
✗ Slow for very large datasets

Slide 47

EXAMPLE OF MULTIPLE LINEAR REGRESSION

Here the matrices for Y and X are givenas follows:

$$X = \begin{Bmatrix} 1\,1\,4 \\ 1\,2\,5 \\ 1\,3\,8 \\ 1\,4\,2 \end{Bmatrix} \quad Y = \begin{Bmatrix} 1 \\ 6 \\ 8 \\ 12 \end{Bmatrix}$$

Weekly sales depend on both Product 1 and Product 2 sales.

$Y=\beta 0+\beta 1X1+\beta 2X2$

Slide 48

EXAMPLE OF MULTIPLE LINEAR REGRESSION

How do we find the coefficients?

Using the normal equation:

$\beta = (X^T X)^{-1} X^T Y$

This formula gives the values of:

$\beta_0, \beta_1, \beta_2$

That minimize the squared error.

$$X^T X = \begin{pmatrix} 1&1&1&1 \\ 1&2&3&4 \\ 4&5&8&2 \end{pmatrix} \begin{pmatrix} 1&1&4 \\ 1&2&5 \\ 1&3&8 \\ 1&4&2 \end{pmatrix} = \begin{pmatrix} 4&10&19 \\ 10&30&46 \\ 19&46&109 \end{pmatrix}$$

$$(X^T X)^{-1} = \begin{pmatrix} 4&10&19 \\ 10&30&46 \\ 19&46&109 \end{pmatrix}^{-1} = \begin{pmatrix} 3.15&-0.59&-0.30 \\ -0.59&0.20&0.016 \\ -0.30&0.016&0.054 \end{pmatrix}$$

$$(X^T X)^{-1} X^T = \begin{pmatrix} 3.15&-0.59&-0.30 \\ -0.59&0.20&0.016 \\ -0.30&0.016&0.054 \end{pmatrix} \times \begin{pmatrix} 1&1&1&1 \\ 1&2&3&4 \\ 4&5&8&2 \end{pmatrix} = \begin{pmatrix} 0.05&0.47&-1.02&0.19 \\ -0.32&-0.098&0.155&0.26 \\ -0.065&0.005&0.185&-0.125 \end{pmatrix}$$

Slide 49

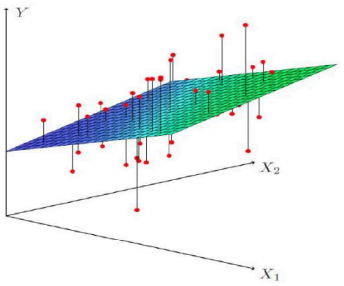
EXAMPLE OF MULTIPLE LINEAR REGRESSION

$$\beta = ((X^T X)^{-1} X^T) Y = \begin{pmatrix} 0.05&0.47&-1.02&0.19 \\ -0.32&-0.098&0.155&0.26 \\ -0.065&0.005&0.185&-0.125 \end{pmatrix} \times \begin{pmatrix} 1 \\ 6 \\ 8 \\ 12 \end{pmatrix} = \begin{pmatrix} -1.69 \\ 3.48 \\ -0.05 \end{pmatrix}$$

$$\boldsymbol{\beta_0 = -1,69}$$
$$\boldsymbol{\beta_1 = 3,48}$$
$$\boldsymbol{\beta_2 = -0,05}$$

Y= -1,69 + 3,48x1+(-0,05)x4

Slide 50



Slide 51

Slide 51: the 3D plane

With 2 inputs, the line becomes a **flat plane** in 3D (x1, x2 on the floor, Y as the height). Red dots = data; black sticks = **residuals**. **Slide 52, "Lab Multiple LR":** your **Lab 2, Task 3.3**.

EXAM Interpret coefficients ("all else constant", sign = direction); build X with 1s; compute X^TX; GD vs normal equation.

Part E: Slides 53–63, polynomial regression

Slides 54–56: the idea and the comparison picture

When the points **curve** (plant growth, temperature over a year, up-then-down prices), a straight line can't follow them. Polynomial regression fits a **curve** using **powers of x**.

- **Left picture, "simple linear model" y = b0 + b1x:** the straight line misses the top dots, which bend upward.
- **Right picture, "polynomial model" y = b0 + b1x + b2x²:** the curve bends with the dots.

⚠ Slide 54: "a simple **polynomial** regression algorithm only works when the relationship is linear" should say simple **linear** regression.

Slides 57–58: the formula and the key trick

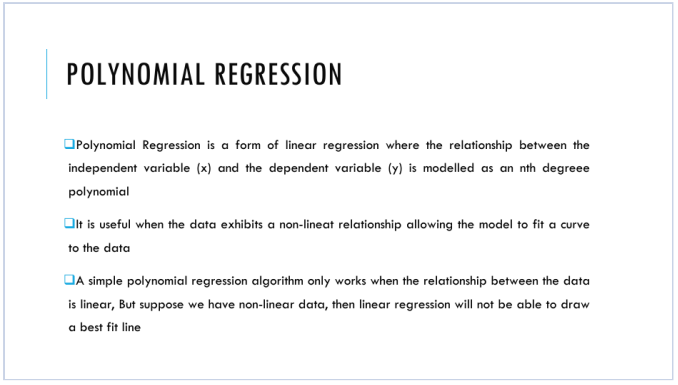
$y = \beta 0 + \beta 1x + \beta 2x^2 + \beta 3x^3 + \ldots + \beta d \cdot x^d + \varepsilon$. d = degree (1 linear, 2 quadratic, 3 cubic); the β's are learned; ε = error.

Key trick: not linear in **x**, but **linear in the β's**. Create the columns x^2, x^3, … and run **ordinary multiple linear regression** on them. That's why it's still "linear", and why GD and the normal equation still work.

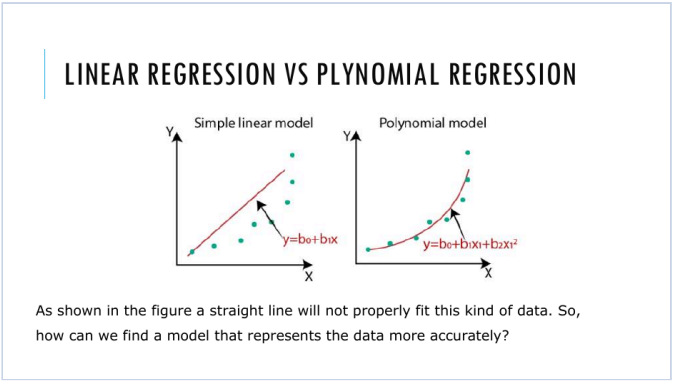
Example: plant growth

Day t	0	1	2	3	4
Height h	1	2	5	10	17

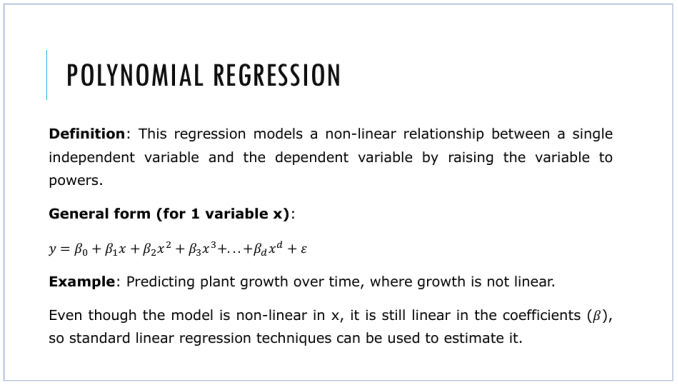
- **Degree 1:** best line h = 4t − 1 → predicts −1, 3, 7, 11, 15 (wrong at both ends), MSE = **2.8**
- **Degree 2:** add a column t^2 (0, 1, 4, 9, 16) → h = **$1 + t^2$** → predicts 1, 2, 5, 10, 17. **Perfect**, MSE = 0.



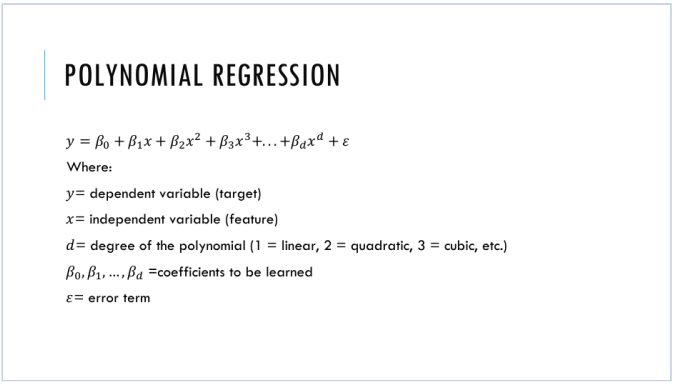
Slide 54



Slide 56



Slide 57



Slide 58

Slides 59–60: degree and types

Degree	Shape	Equation
1	straight line	y = β0 + β1x
2	parabola, 1 bend	y = β0 + β1x + β2x²
3	2 bends	+ β3x³
4+	more bends	quartic, …

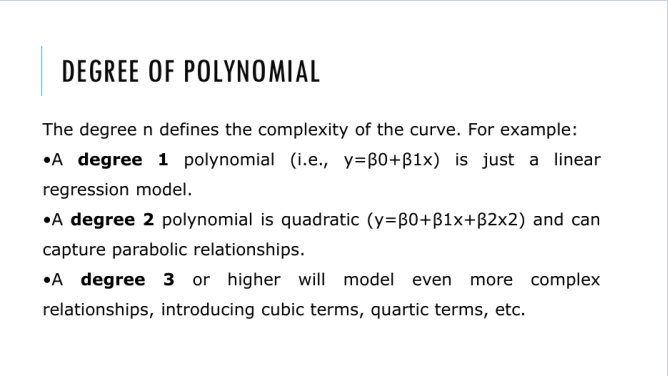
Slide 61: degrees 1–4 on wavy data

The orange dots go up, down, up, down. **Green (deg 1):** a straight line that misses everything (underfitting). **Blue (deg 2):** one bend, still poor. **Red (deg 3):** follows part of the wave. **Yellow (deg 4):** follows the wave best. Too high a degree (e.g. 10) wiggles through every point, memorizes the noise and predicts new data badly. That's **overfitting**, the opposite of generalization (Ch. 1). Choose the degree using the error on the **test** set. (Lab 2, Task 3.4 asks this.)

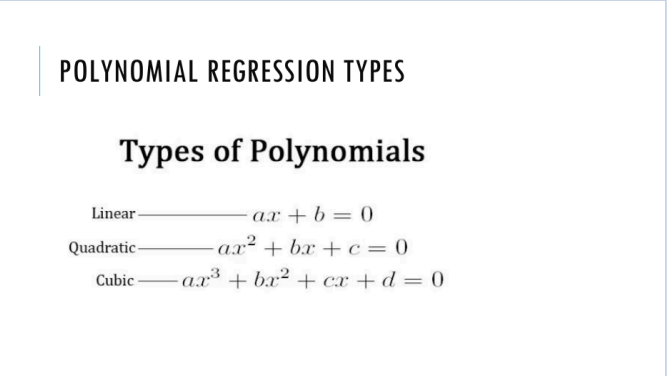
Slide 62: when to use it

1. **Curved trends:** plant growth; population growth that speeds up or slows down.
2. **Seasonal / cyclical patterns:** temperature over the year; sales or stock cycles.
3. **Turning points:** y increases then decreases (or vice versa).

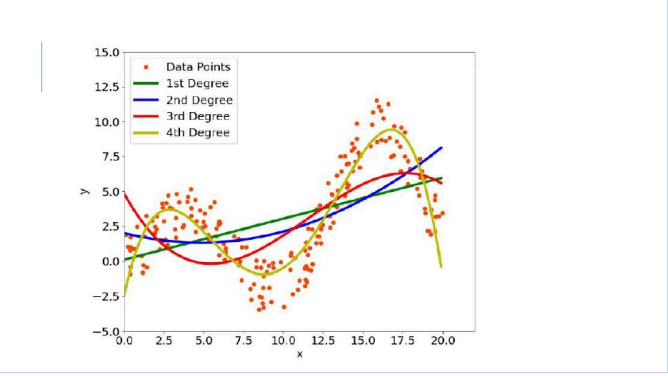
Slide 63, "Lab Polynomial Regression": your **Lab 2, Task 3.4**.



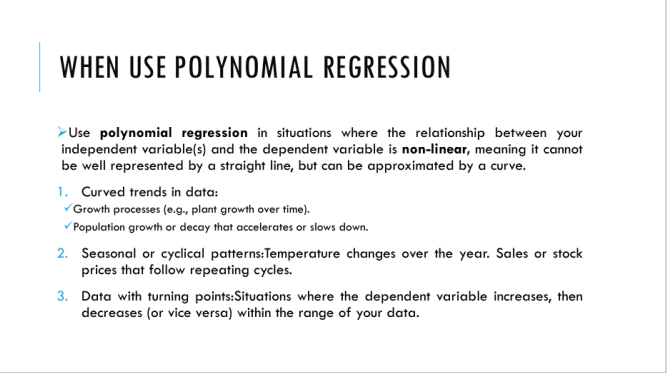
Slide 59



Slide 60



Slide 61



Slide 62

EXAM Why it's still "linear" (linear in β); which degree fits a picture; overfitting risk; the 3 situations to use it.

⚠ All slide errors in slides 14–63

Slide	It says	Correct
14	"close to the **predicted** value"	close to the **actual** value
39	"MAE … squares the difference"	MAE = **absolute value**; MSE squares
49	$(X^TX)^{-1}X^T$ first entry **0.05**	**1.36**
50	$Y = -1.69 + 3.48x1 - 0.05$**x4**	… − 0.05**x2**
54	"simple **polynomial** regression only works when linear"	simple **linear** regression

Exam priority checklist

★	Topic	Slides
★★★	Gradient descent iterations by hand	35–37
★★★	Derivative formulas + update rule	31–33
★★★	Simultaneous update: correct vs incorrect	31
★★★	MAE / MSE / RMSE calculations	39–40
★★	J(w) for several w values, U curve	19–21
★★	α too small / too large	38
★★	Slope > 0 → w decreases; < 0 → w increases	34
★★	Multiple LR: interpret coefficients, normal equation steps	44–50
★★	Polynomial: linear in β, degree, overfitting	54–62
★	Contour plots, local vs global minimum, $(\bar{x}, \bar{y})$	15, 23–26, 30

Answer to the iteration 2 practice: errors -0.1, -0.675, -0.25, -1.825 → $\partial J/\partial w = 1/4 \cdot (-0.1 - 1.35 - 0.75 - 7.3) = \mathbf{-2.375}$, $\partial J/\partial b = 1/4 \cdot (-2.85) = \mathbf{-0.7125}$ → $w = 1.425 + 0.2375 = \mathbf{1.6625}$, $b = 0.475 + 0.07125 = \mathbf{0.54625}$, $J \approx \mathbf{0.1239}$.

The whole chapter in one chain: labelled data → $\hat{y} = wx + b$ → residuals → cost J (MSE) → gradient descent with a good α → several inputs: multiple LR (GD or normal equation) → curves: polynomial LR (watch for overfitting) → evaluate on test data with MAE / MSE / RMSE.